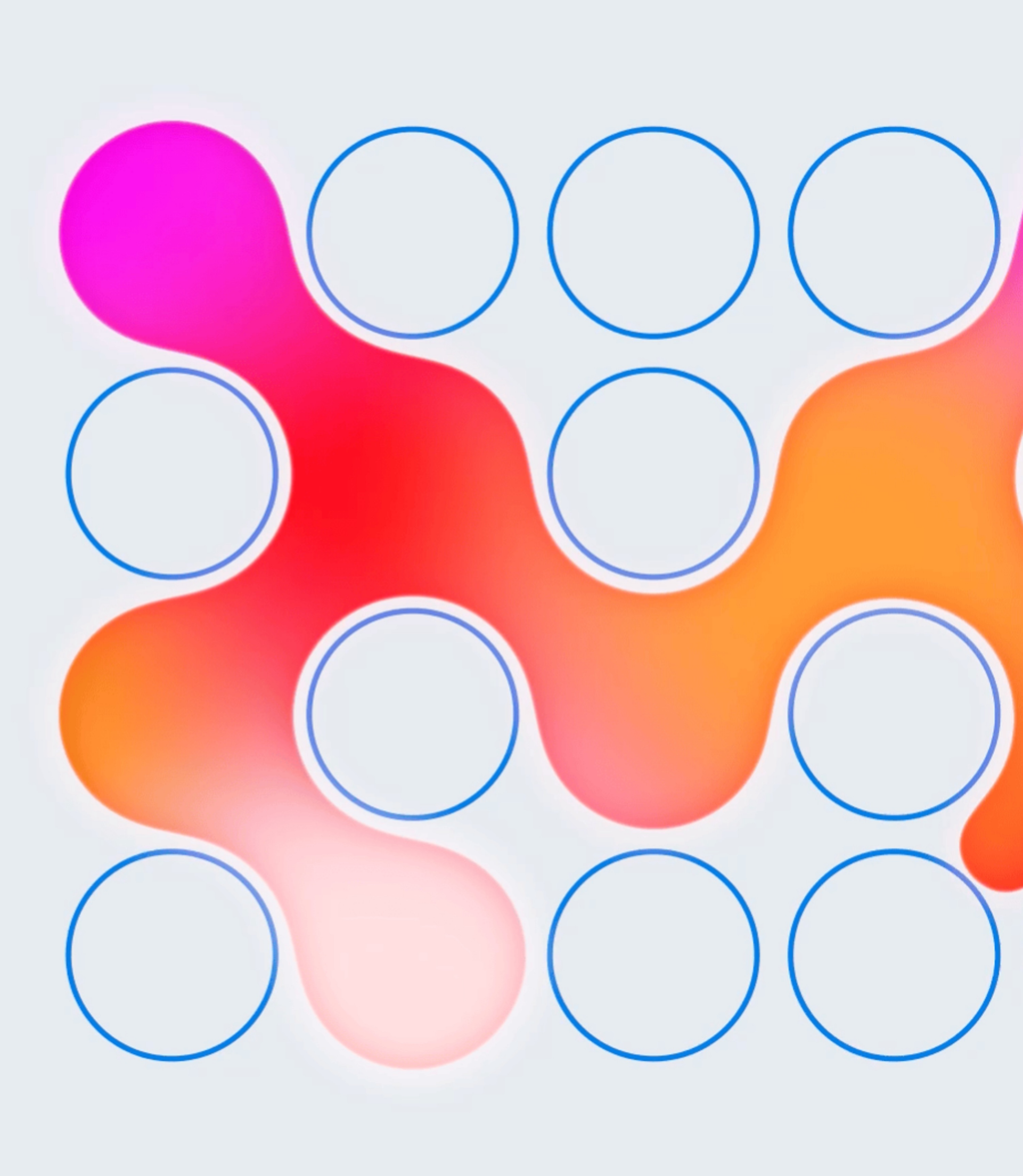




Building Agentic Systems on AWS

Domain specific agent applications



Presenters

Leonardo Murillo

Developer Relations,
AWS Marketplace



Thaddeus Worsnop

Principal, Solutions Architect,
Amazon Web Services (AWS)



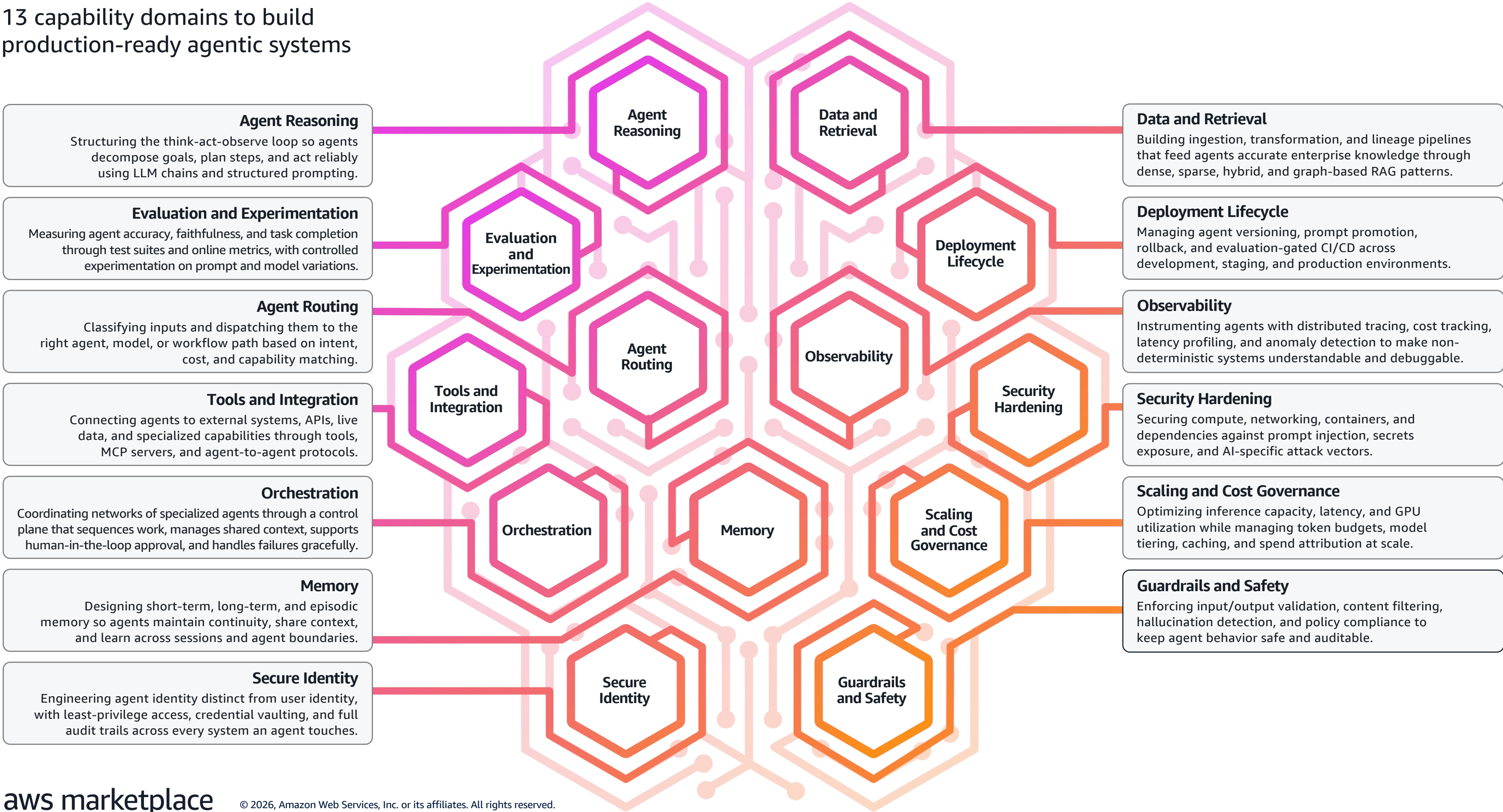
Agenda

- The domain specialization problem
- Adapting the architecture
- Domain-specific agent examples
- Grounding in knowledge (RAG for domains)
- Guardrails and constraints
- Domain-specific evaluation
- Integrating into multi-agent systems
- Demo
- Next steps



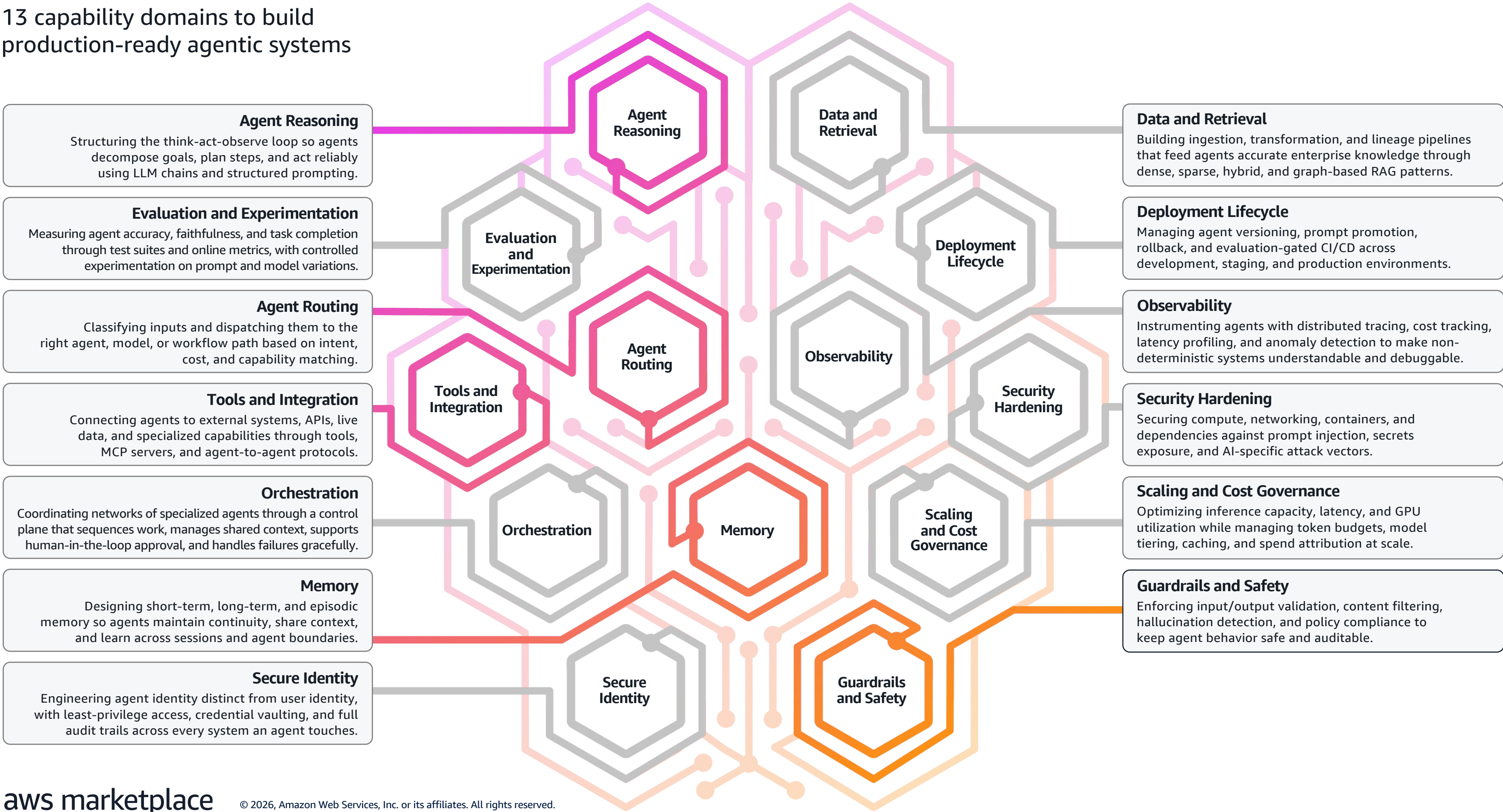
The Agentic AI capability stack

13 capability domains to build production-ready agentic systems



The Agentic AI capability stack

13 capability domains to build production-ready agentic systems



Agentic AI tools and capabilities in AWS Marketplace

AGENT REASONING

Claude

cohere

Hugging Face

LangChain

Dify

Progress Agentic RAG

composable

Amazon Bedrock

STRANDS AGENTS

EVALUATION & EXPERIMENTATION

LangChain

Langfuse

vellum

Weights & Biases

comet LLM

Patronus AI

deepchecks.

Amazon Bedrock AgentCore (Evaluations)

Amazon SageMaker AI

Amazon CloudWatch

AGENT ROUTING

LangChain

AWS Step Functions

Amazon EventBridge

Amazon Bedrock

TOOLS & INTEGRATION

algolia

mapbox

Deepgram

Kong Connect

Tyk

TAILWINDS

portkey

SMARTBEAR

krakenD

AWS Lambda

Amazon API Gateway

Amazon Bedrock AgentCore (Gateway)

ORCHESTRATION

Temporal

orkes

PREFECT

ASTRONOMER

AWS Step Functions

Amazon Bedrock Agents

Amazon MWA

MEMORY

zilliz

redis

elastic

TiDB Cloud

MongoDB Atlas

yugabyteDB

Pinecone

Weaviate

Amazon OpenSearch

Amazon MemoryDB

Amazon Bedrock AgentCore (Memory)

SECURE IDENTITY

Ping Identity

okta

CYBERARK

Amazon Cognito

AWS Secrets Manager

Amazon Bedrock AgentCore (Identity)

DATA & RETRIEVAL

CONFLUENT

databricks

Fivetran

Collibra

Alation

Bigeye

MONTE CARLO

boomi

Amazon Bedrock Knowledge Bases

Amazon S3

AWS Glue

DEPLOYMENT LIFECYCLE

Pulumi

GitLab

Vercel

LaunchDarkly

Vault

KNIME

AWS CodePipeline

Amazon SageMaker AI

AWS CDK

OBSERVABILITY

DATADOG

honeycomb.io

arize

LangWatch

True Watch

INSTANA

logz.io

Amazon CloudWatch

AWS X-Ray

AWS Distro for OpenTelemetry

SECURITY HARDENING

snyk

Protect AI

Prompt

tailscale

Amazon GuardDuty

AWS Security Hub

Amazon Inspector

SCALING & COST GOVERNANCE

ClickHouse

NVIDIA AI ENTERPRISE

Modal

cast.ai

baseten

anyscale

hydrolix

StormForge

Amazon EKS

AWS Fargate

Amazon Bedrock AgentCore (Runtime)

GUARDRAILS & SAFETY

Claude

Guardrails AI

Arthur

Amazon Bedrock Guardrails

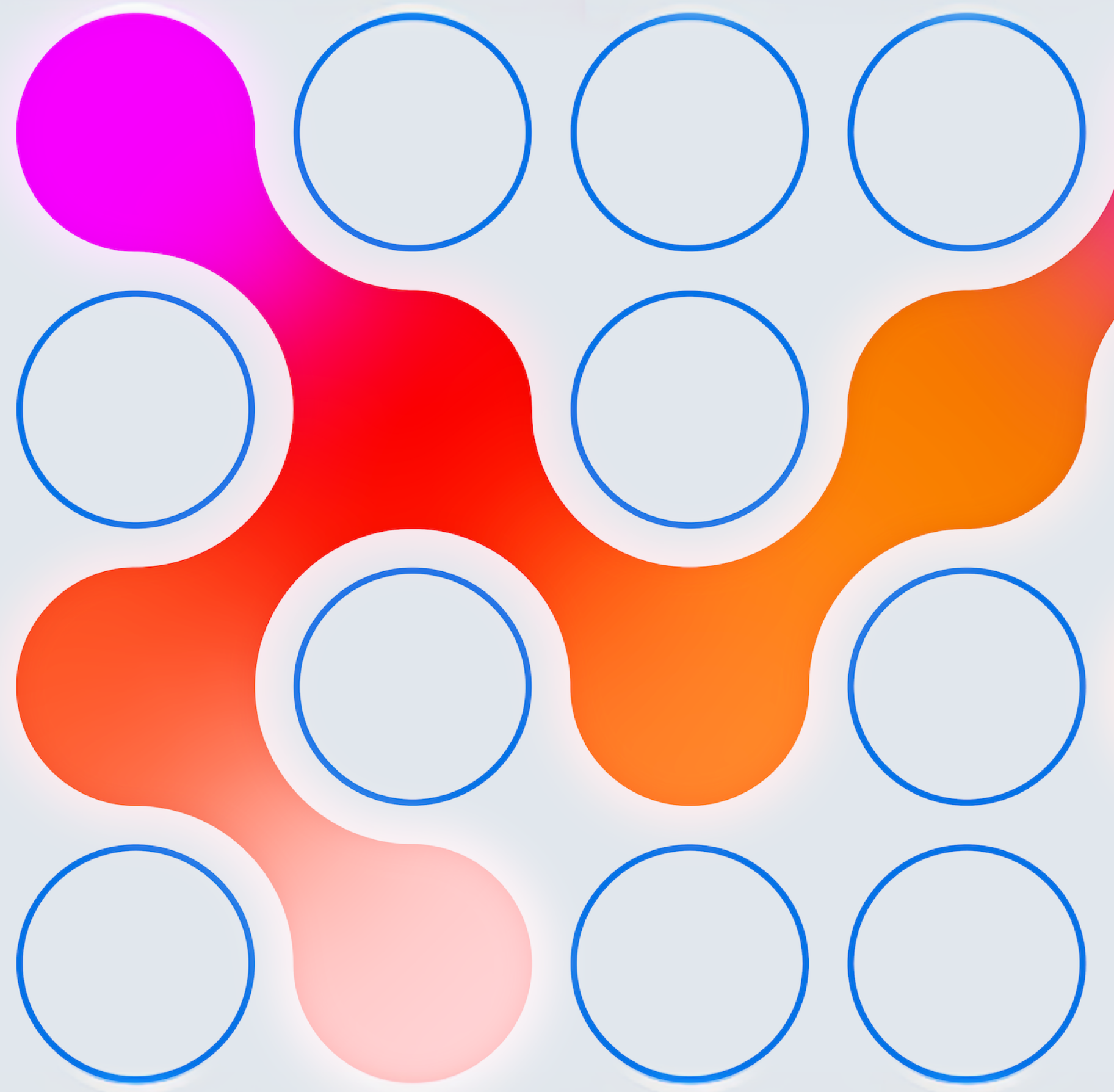
Amazon Bedrock AgentCore (Policy)

2,800+

Agentic AI tools and capabilities

To help you build, deploy, and operate agentic systems on AWS—available through your AWS account.

Domain specialization



Does general capability translate to domain reliability?

The reliability gap: where a capable generalist becomes an unreliable specialist

General capability

What the base model gives you out of the box — breadth of reasoning.

Coverage	Broad — trained across many domains
Reasoning	Coherent across most topics
Consistency	Variable — acceptable on common inputs
Edge cases	Silent failure modes, subtly wrong output
Threshold	Typically 80–90% on unspecialized tasks

Domain reliability

What production needs — consistent correctness in this specific context.

Coverage	Depth on the target domain only
Reasoning	Channeled through domain-specific patterns
Consistency	Stable across the full input distribution
Edge cases	Explicit handling, measured via evaluation
Threshold	90–95% customer-facing ~99% safety-critical

▶ **Prescriptive:** Specialization is not restriction — it is focusing general reasoning on a domain through four deliberate levers.

The four levers of domain adaptation

Every domain specialization decision operates on one of these four levers

System prompt

Primary mechanism

Encodes domain persona, priorities, refusals, uncertainty handling, and communication norms in four layers: role, knowledge context, constraints, communication.

Overridable by adversarial input · not sufficient alone

Knowledge corpus

Retrieval grounding

Domain documents, facts, and policies the agent retrieves at inference via Amazon Bedrock Knowledge Bases. Quality bounded by relevance, accuracy, freshness, and coverage.

Curation effort matches prompt engineering effort

Tool selection

Capability scoping

Which MCP-exposed tools the agent can invoke. Scoping improves quality (no inappropriate tools for domain tasks) AND security (limits blast radius).

Routing tools ≠ deployment tools · scope ruthlessly

Guardrails

Unconditional enforcement

What the agent must never do in this domain. Enforced at Amazon Bedrock Guardrails, Patronus AI, or Deepchecks layer – NOT in the system prompt.

Must survive model reasoning attempts to override

▶ **Rule of thumb:** If it affects quality, tune the system prompt and corpus. If it affects safety or liability, enforce it in Guardrails.

Domain reliability thresholds

Set your target BEFORE you build—the threshold dictates corpus depth, evaluation size, and rollout policy

80–85%

Internal productivity tools

Users can identify and correct errors; low cost of inconsistency.

Examples: Internal DevOps companion · code-review assistant · drafting helper

90–95%

Customer-facing interactions

Errors erode trust, often cannot be undone after the fact.

Examples: Customer engagement & sales agents · support automation · onboarding flows

~99%

Safety-critical domains

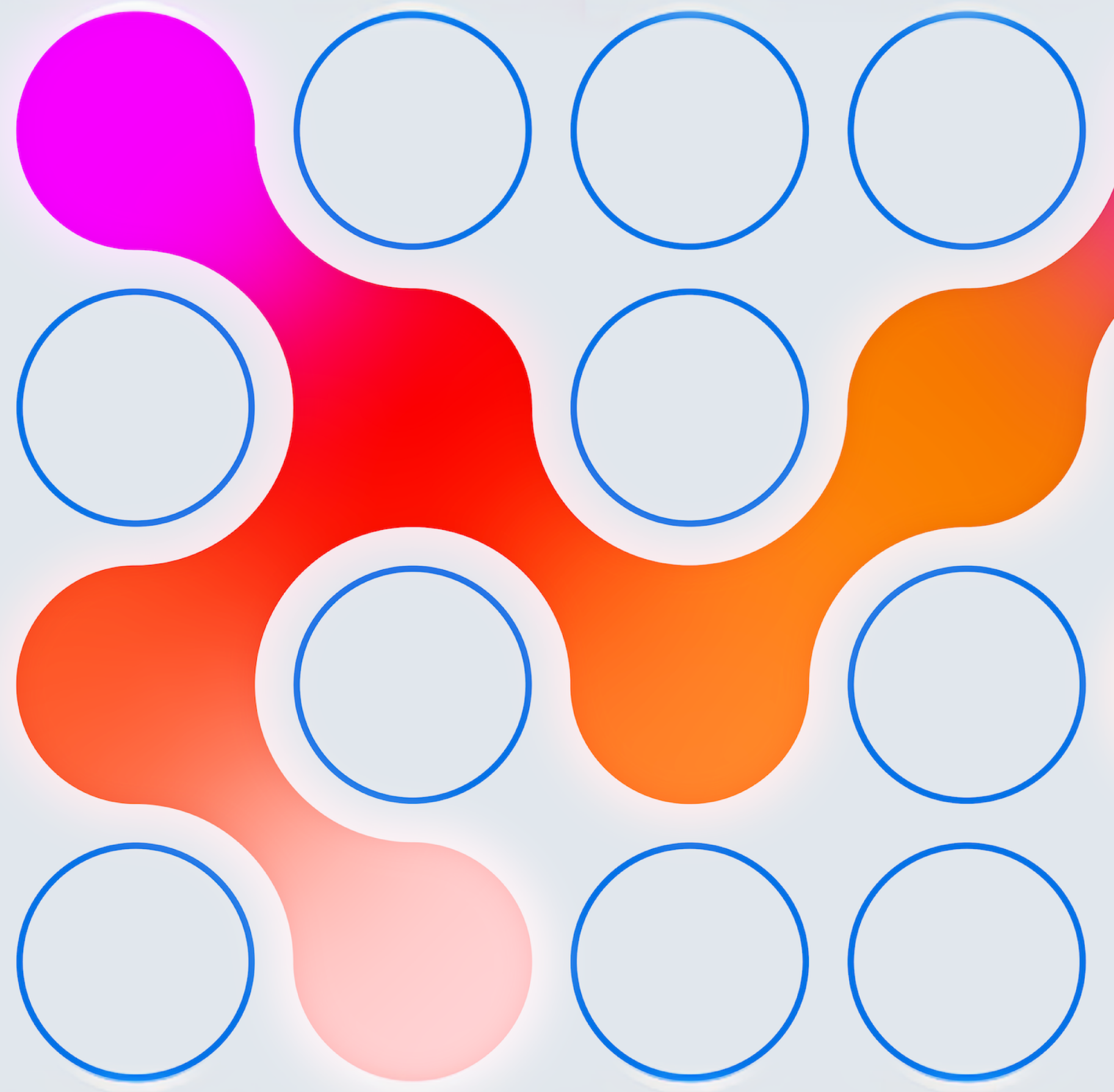
Incorrect output has legal, medical, or financial consequences.

Examples: Medical information · personalized finance · legal drafting · safety-related routing



Measured, not aspirational: Thresholds must be observed on a domain-representative golden dataset — not on a hand-picked demo set.

Adapting the architecture



Domain system prompt: a four-layer contract

General prompts set guidelines. Domain prompts establish identity, knowledge context, constraints, and communication norms.

LAYER 1

Role and authority

Who the agent is, what it is responsible for, what authority it holds in this domain. Defines the subject of every downstream decision.

EXAMPLE

"You are the onboarding specialist for the DevOps Companion platform. You capture requirements; you do not provision infrastructure."

LAYER 2

Knowledge context

What the agent knows, which corpora to consult for what, how to handle gaps. Declare the retrieval strategy in the prompt itself.

EXAMPLE

"For capability questions use the capability KB. For pricing consult the product catalog. If a term is absent from both, say so — do not fabricate."

LAYER 3

Behavioral constraints

What the agent must always do, must never do, and how to handle uncertainty. Complemented by—but not a replacement for—guardrails.

EXAMPLE

"Never quote a price that is not directly retrieved from the catalog tool. Never commit to delivery dates — refer to the logistics agent."

LAYER 4

Communication norms

Tone, format, length constraints, escalation procedure. Interface-specific—voice prompts constrain length much more tightly than text.

EXAMPLE

"Voice mode: 2–4 sentences per response. No bullet points or code blocks. End with a concrete offer of next step."

Knowledge corpus design for domain agents

Three criteria · Amazon Bedrock Knowledge Bases · Hierarchical chunking at semantic boundaries

Relevance

Corpus contains primarily information directly useful for the agent's tasks. Irrelevant content reduces signal-to-noise in semantic search.

How: Audit corpus quarterly · remove stale categories · measure % of retrieved chunks used in responses

Freshness

Corpus updated when domain facts change: prices, policies, geospatial data, regulatory requirements. Staleness is silent—retrieval does not flag it.

How: Tag every document with `updated_at` · fail ingestion when age > 30d for price/policy docs · pipeline alerts on staleness

Coverage

Corpus addresses the full range of queries encountered, including edge cases and long-tail. Most RAG failures are "no relevant chunk found," not "wrong chunk retrieved."

How: Log low-confidence retrievals · cluster into coverage-gap themes · commission domain experts to author fill-in content

Chunking strategy by document type

DOCUMENT TYPE

Narrative / policy docs

Structured data (catalog, routes, configs)

Long hierarchical manuals

STRATEGY

Semantic chunking at paragraph or section boundaries

Record-level chunks (1 record = 1 chunk)

Hierarchical chunking via Amazon Bedrock Knowledge Bases

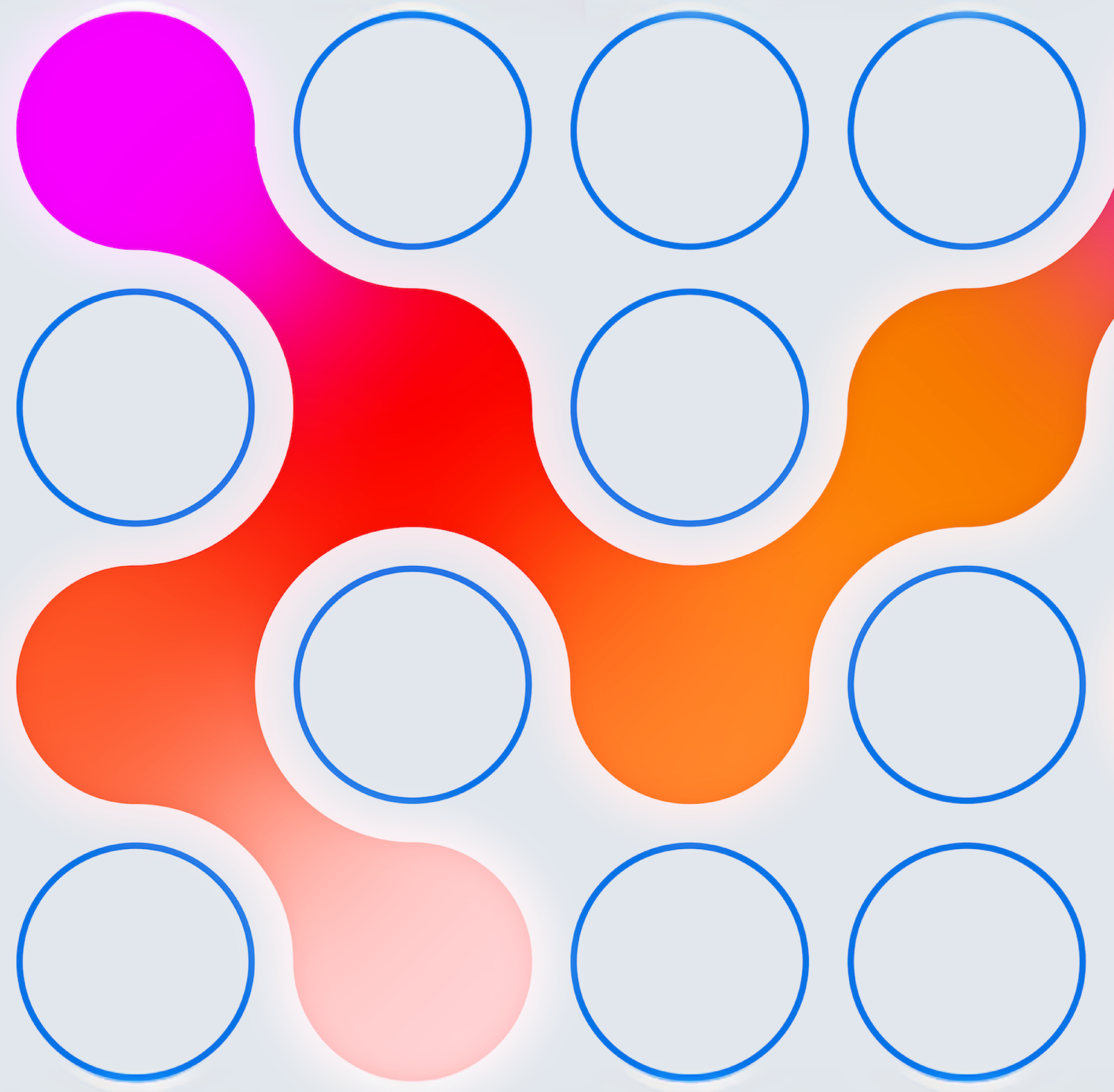
WHY

Preserves complete, self-contained statements

Product A chunks never contaminated with Product B data

Parent-child links surface fact AND broader context

Customer engagement and sales agents



Intent as the entry point

Every customer interaction begins with a classification problem—the answer determines retrieval, tools, tone, and scope

Customers do not state intent precisely

"My order is wrong."

Possible intents:

- Refund for the incorrect order
- Replacement of the correct item
- Apology acknowledging the mistake
- Information about what went wrong
- Combination of any of the above

The agent must infer likely intent from context—conversation history, account, recent transactions—and remain responsive to clarification.

Prescriptive: choose your intent layer

Structured flows

Amazon Lex + Amazon Connect

When: When the input surface is narrow: menu-driven customer paths, slot filling (date · account · product SKU), contact-center IVR.

Why: Deterministic. Debuggable. Integrates natively with Connect agent workspace for seamless voice-to-chat handoff.

Open-ended natural language

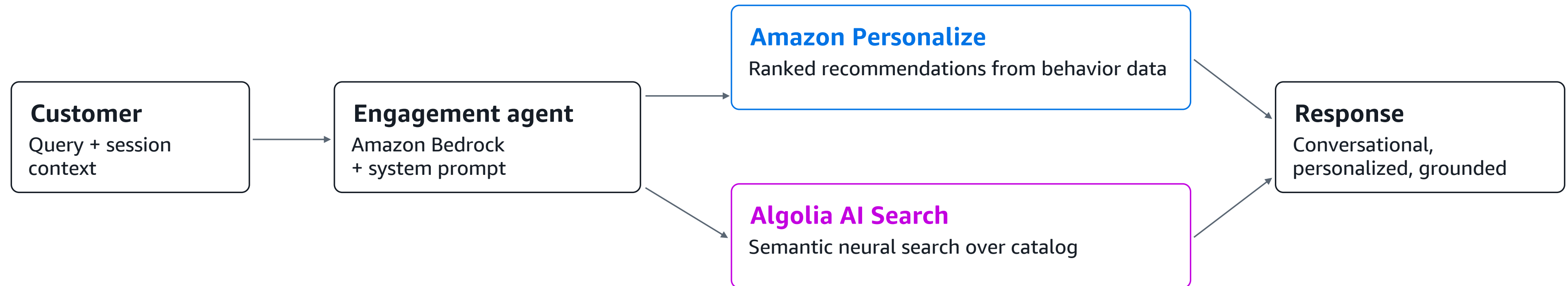
Amazon Bedrock agents + retrieval

When: When customers describe problems in their own words, use free-form queries, and expect nuanced responses.

Why: A well-designed system prompt and grounding corpus beats rule-based classifiers on natural queries.

Personalization at scale

Amazon Personalize and Algolia AI Search—two complementary surfaces that free the agent from reasoning both about recommendation AND retrieval from scratch



Amazon Personalize

The agent does not reason from scratch about what to recommend. It queries a Personalize endpoint trained on actual customer behavior and receives a ranked list. Agent reasoning then selects and contextualizes — not invents.

Algolia AI Search

High-performance semantic search over product catalogs and content libraries. Surfaces long-tail and ambiguous queries that keyword search would miss, so the agent can respond with genuinely relevant product information.

Available in AWS Marketplace.

Escalation design

No customer engagement agent ships without a handoff policy—three trigger categories and context-preserving transfer via Amazon Connect

Scope escalation

Customer request falls outside the agent's authorized scope.

DETECTION SIGNALS

- Intent classification confidence high
- Intent matches an out-of-scope category
- Example: legal commitment, pricing override

Sentiment escalation

Customer tone indicates distress, frustration, or dissatisfaction requiring a human.

DETECTION SIGNALS

- Sentiment analysis on latest turn
- Repeated negative sentiment across turns
- Explicit distress keywords

Complexity escalation

Agent confidence in its response falls below the domain reliability threshold.

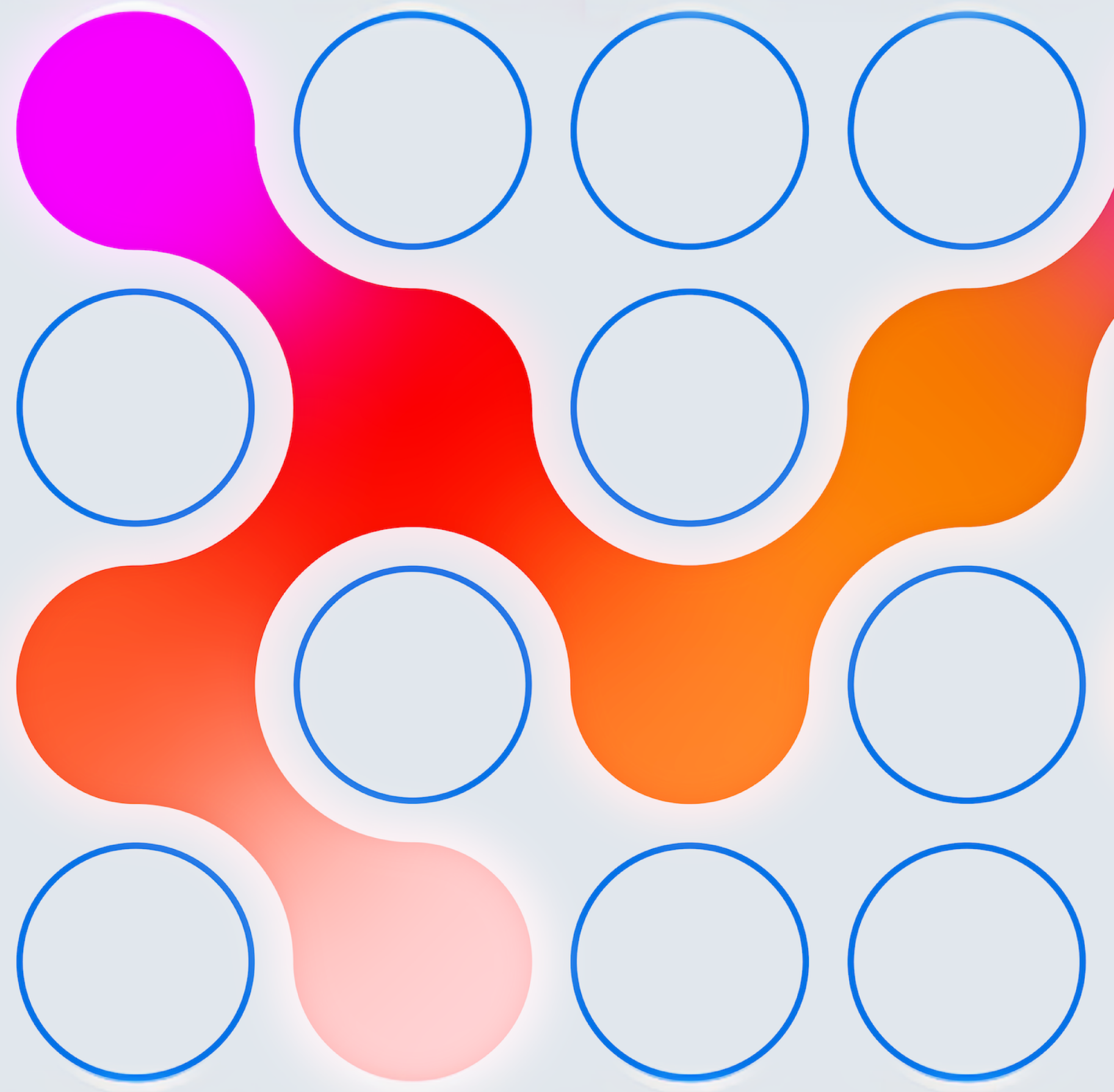
DETECTION SIGNALS

- Retrieval confidence below threshold
- Generation self-consistency check failed
- Multi-turn reasoning loop exceeded

Context-preserving handoff via Amazon Connect

When escalation fires, the agent passes the full conversation context—issue classification, sentiment, account data—to the Connect agent workspace. The human continues seamlessly. The customer never repeats themselves. This single capability is often the largest measurable quality gain agentic systems bring to contact-center operations.

Location-aware agents



Why location agents need tools, not training

Language models know where Paris is. They cannot compute the optimal route among twelve stops with time-window and capacity constraints.



What language models know spatially

Static geography: Paris is in France. Manhattan is an island. The Amazon flows east. Useful for cross-reference, useless for real-time reasoning.

Out of date: No traffic conditions · no live delivery routes · no precise GPS coordinates for a specific address · no current road closures



What language models cannot do spatially

Combinatorial optimization: "Optimal route among twelve stops with time windows and vehicle capacity" is a TSP variant. Simple algorithmic implementations handle it orders of magnitude more reliably than model inference.

Precise spatial math: Haversine distances, polygon containment, geofence intersections—math, not reasoning. Use the purpose-built API.

THE CORE DESIGN PRINCIPLE

Use the model for **natural language understanding, intent interpretation, and result synthesis**. Use geospatial tools for **all spatial data access and computation**. The boundary must be explicit and enforced through the tool schema.

Geospatial tool stack for location agents

Amazon Location Service as foundation · Mapbox (AWS Marketplace)
for advanced routing · each exposed through MCP

Amazon Location Service

Foundation — place search, geocoding, routing, geofencing, map rendering

Place search	Addresses, POIs, coordinates from natural-language descriptions
Geocoding	Bidirectional address ↔ coordinate conversion
Route calc	Optimal path between multiple waypoints, subject to constraints
Geofencing	Detect when a tracked entity enters/exits a defined area
Map rendering	Generate map visualizations for user-facing output

Mapbox Location Services *Available in AWS Marketplace.*

Advanced routing + real-time traffic · complements the AWS foundation

Traffic routing	Live traffic, road closures, turn restrictions baked in
Multi-modal	Driving, walking, cycling, public transit in one API
Day-long routes	Accounts for traffic that evolves across the delivery day
High-res maps	Cartographic-quality visualizations for driver tools
MCP integration	Routing and search exposed as standard agent tools

Freshness and caching in geospatial contexts

Not all geospatial data has the same freshness requirement—cache the slow-moving, fetch fresh for the real-time

Static—cache aggressively

EXAMPLES

Administrative boundaries ·
Major road networks · Regional
topology · Place metadata

STRATEGY

Amazon ElastiCache
TTL days to weeks · Refresh
on release schedule

*Cost per fresh fetch vs. staleness
risk strongly favors caching*

Semi-static—cache with care

EXAMPLES

Pre-computed frequent routes ·
Place search results for
common queries

STRATEGY

Amazon ElastiCache · TTL
minutes to hours · Invalidate
on known triggers

*Set TTL based on the cost of a
stale result in your domain*

Real-time—never cache

EXAMPLES

Live traffic · Vehicle locations ·
Delivery status · Weather
affecting routes

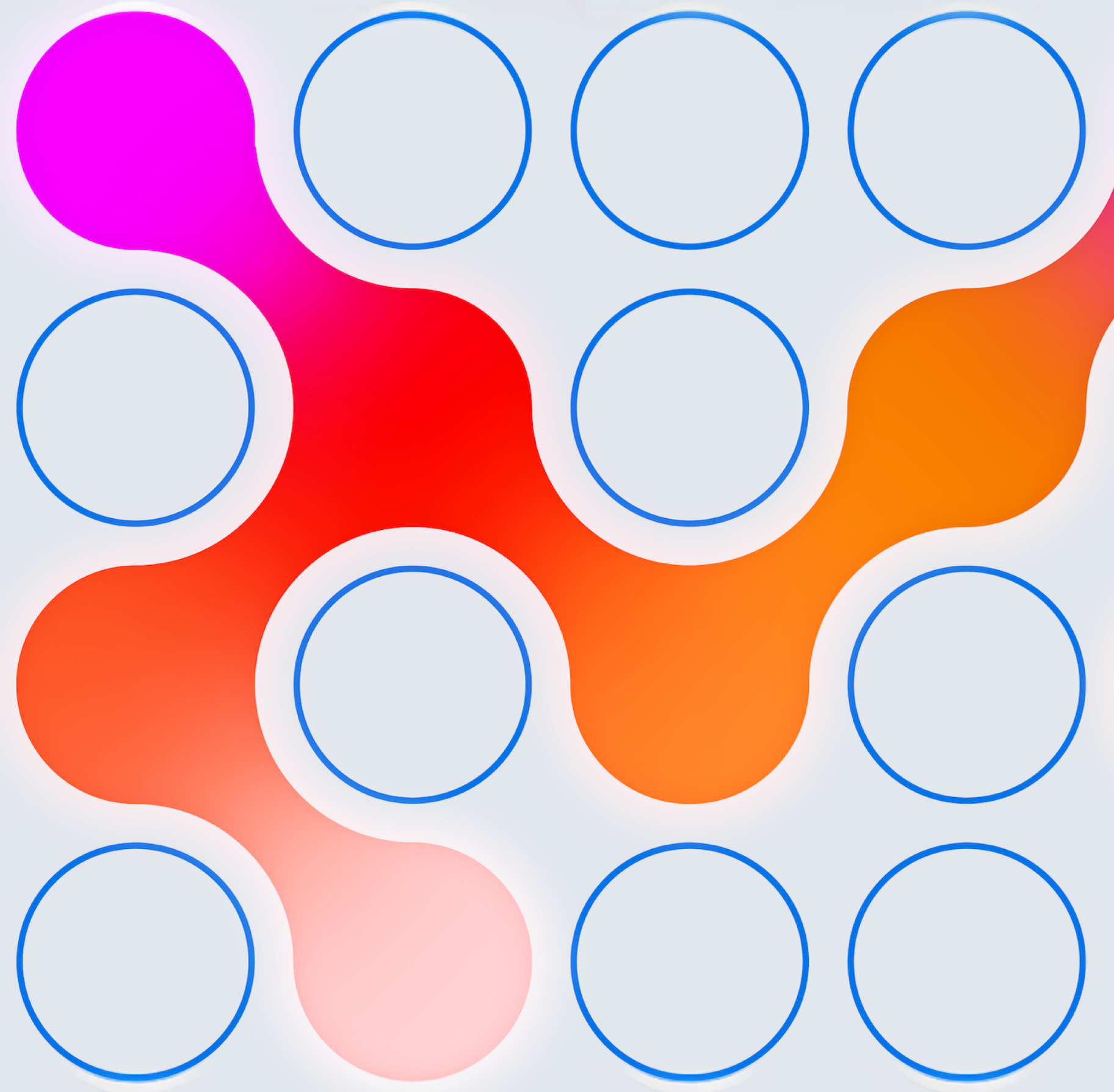
STRATEGY

Direct API fetch every
invocation · No
intermediate cache

*Even a 5-minute stale traffic
snapshot sends a driver into a
jam that formed 3 minutes ago*

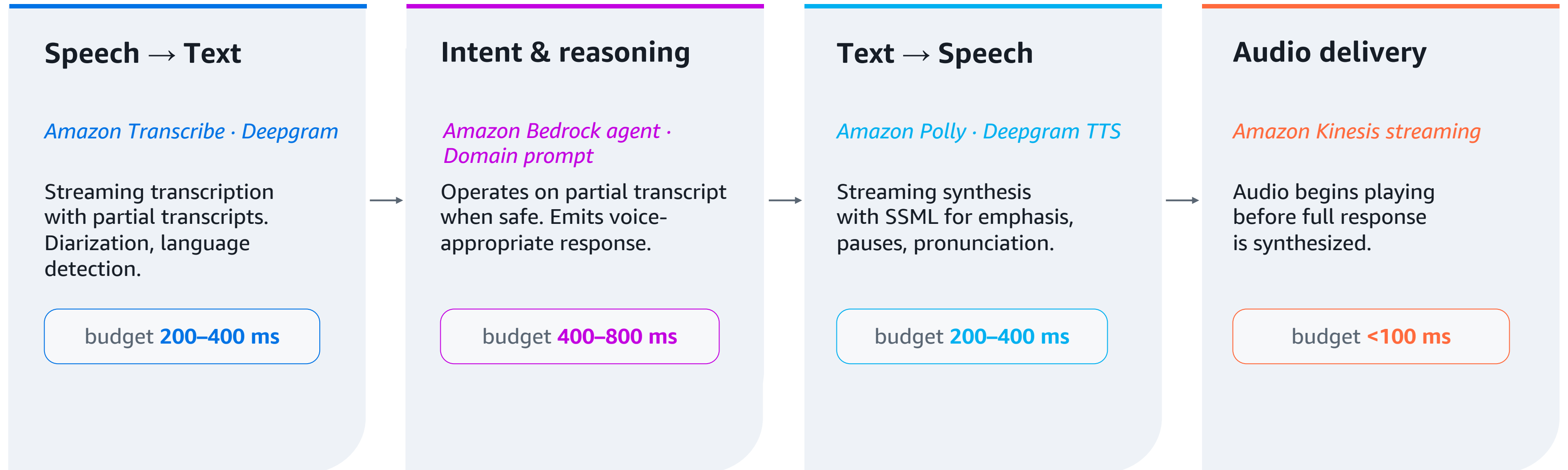
▶ **Do not cache traffic data.** The cost of a stale cache hit here is operational, not just cosmetic — a driver sent into a fresh jam.

Voice-enabled agents



The voice-enabled agent pipeline

Four stages · Streaming at every boundary · Total perceived latency $\leq 1\text{--}2$ seconds



PERCEIVED LATENCY TARGET



< 1 second from end-of-utterance to start-of-response
Streaming at every boundary (partial transcripts, overlapped synthesis, Kinesis audio) collapses sequential stages into a pipeline that begins speaking before reasoning is complete.

Designing for spoken interaction

Voice is not "text with a speaker"—agent reasoning, response shape, and error handling must all be voice-native

Responses must be concise

2–4 sentences. Give the direct answer, then offer more.

EXAMPLE

"The deployment completed successfully to us-east-1. Would you like me to walk through the observability configuration?"

Clarifications must be specific

Ask a concrete choice question, not "I didn't understand."

EXAMPLE

"Did you mean the production environment or the staging environment?"

Handle interruptions gracefully

Detect barge-in. Stop TTS. Incorporate the interruption's context into the next turn.

EXAMPLE

User interrupts mid-response → stop playback → treat new utterance as continuation, not restart.

Multilingual is end-to-end

STT, system prompt, corpus, AND TTS all language-matched. Responding in English to Spanish STT defeats the purpose.

EXAMPLE

Declare supported languages in prompt · separate corpus per language · matched TTS voice model.



Deepgram Voice AI: Nova-2 STT delivers streaming latency in tens of milliseconds. MCP server exposes STT and TTS as tool calls—same pattern as any agent tool. Available in AWS Marketplace.

Domain-specific guardrails—enforce at infrastructure, not in the prompt

System prompt constraints can be talked around. Amazon Bedrock Guardrails, Patronus AI, and Deepchecks enforce unconditionally.

Customer engagement

Price guarantees

Never commit to prices not verified in real time from catalog

Regulated disclosures

Require specific disclosure text when mentioning regulated products

PII detection

Built-in PII redaction on inputs AND outputs via Amazon Bedrock Guardrails

Authority limits

Hard block on commitments outside authorized scope

Sentiment escalation

Automatic human routing on distress signals

Location and logistics

Confidence thresholds

Present as estimate when route confidence < threshold

Data freshness

Max age of geospatial data enforced per tool type

Coverage boundaries

Block recommendations in areas with inadequate data coverage

Liability disclaimers

Auto-append for recommendations with significant uncertainty

Safety routing

Never recommend against local traffic laws or restrictions

Voice interface

Format enforcement

Block markdown, HTML, code blocks in voice mode

Length caps

Max response words/duration per interaction turn

Language match

Block cross-language responses unless explicitly requested

Interrupt safety

Require safe handling of partial commands in interruption

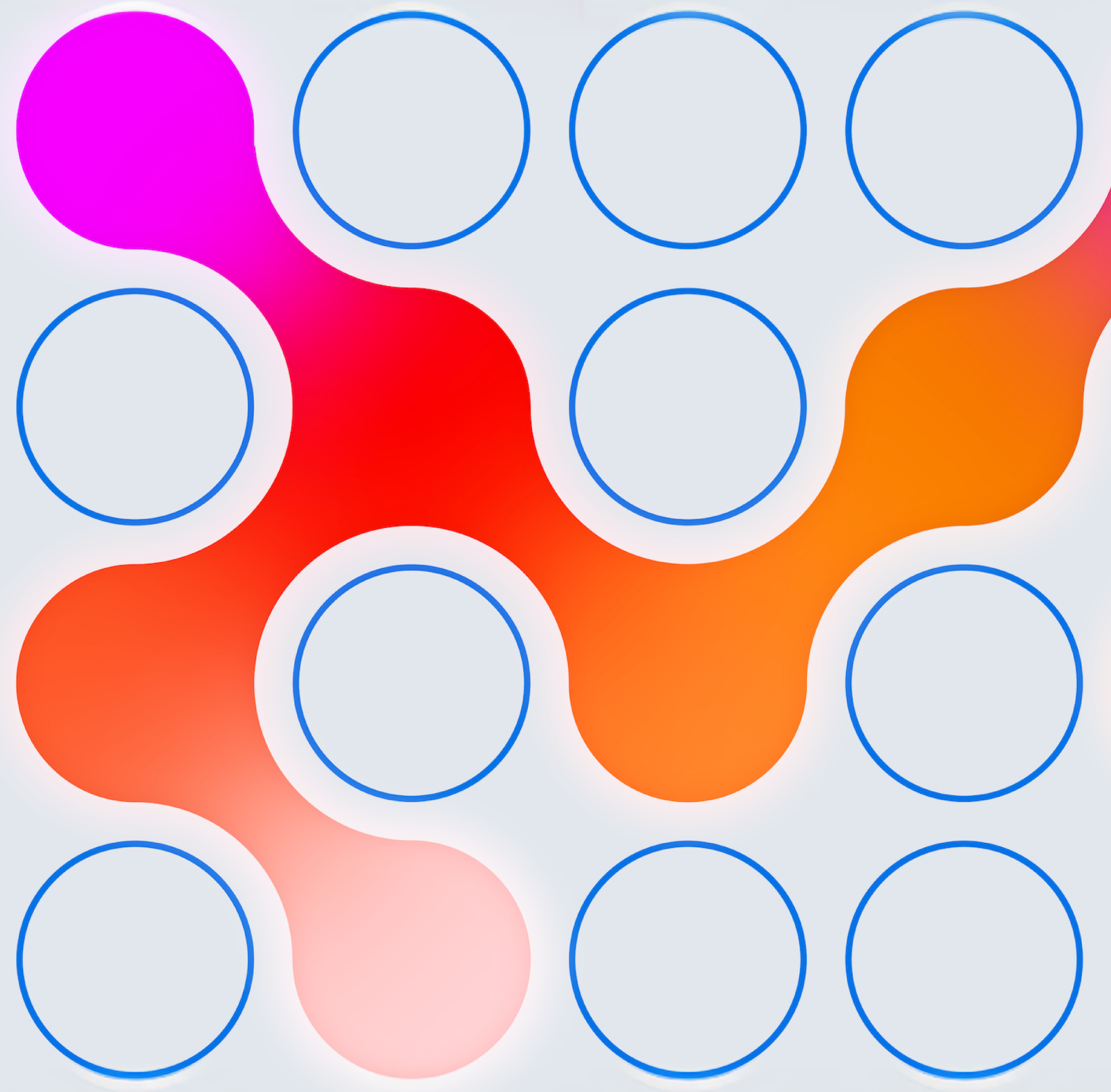
Transcription confidence

Auto-clarify when STT confidence below threshold



Key principle: If it affects liability, compliance, or safety—it does not live in the system prompt. It lives in Amazon Bedrock Guardrails where model reasoning cannot override it.

Domain specific evaluation



Domain-specific evaluation—beyond generic quality signals

Amazon Bedrock AgentCore Evaluations · custom evaluators · domain experts in the loop

Build a domain-representative golden dataset

Drawn from the actual input distribution, not engineers' assumptions

- Pull from real customer support tickets, chat logs, call recordings, routing requests
- Include common requests, common edge cases, AND common failure modes
- Do NOT hand-construct from what engineers think users ask
- Annotate with domain-expert involvement

Automate the high-volume layer · human-review the edges

LLM-as-judge with domain rubrics + schema validation + automated fact-checking

- Automated evaluators run continuously on every deployment and every N% of production traffic
- Flag responses below threshold on objective criteria
- Human experts review flagged responses and contribute annotations back
- Amazon Bedrock AgentCore Evaluations supports custom evaluators for domain-specific rubrics

Close the loop — annotations become new test cases

The evaluation system improves as the agent is used in production

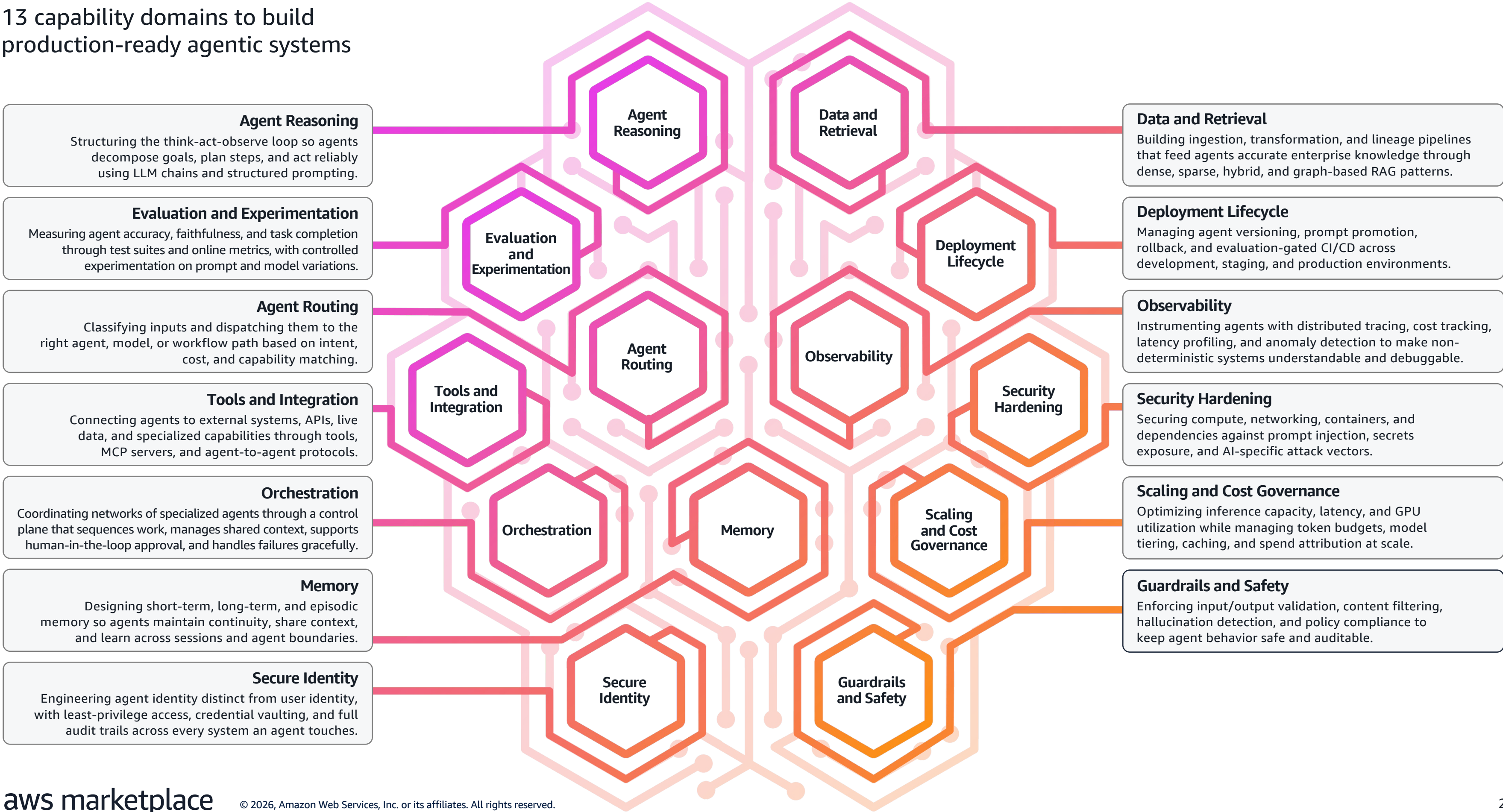
- Every expert annotation is a candidate test case
- Production failures surface new evaluation dimensions
- Golden dataset grows along the axes that actually matter
- Quality gate tightens over time without manual spec rewrites



Common pitfall: A general agent at 92% on a hand-picked set can fall to 75% on the domain-representative distribution. That gap is a production incident.

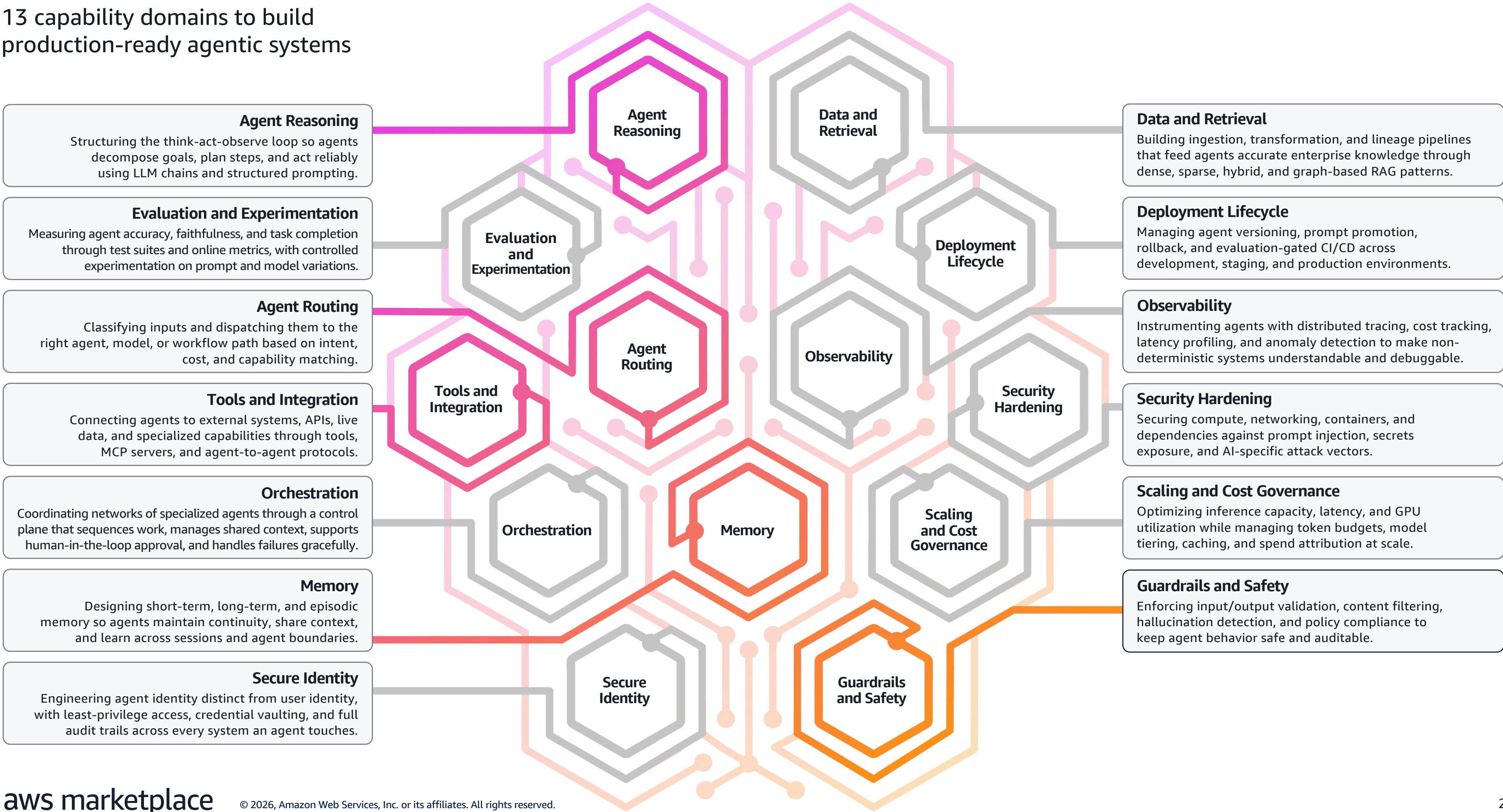
The Agentic AI capability stack

13 capability domains to build production-ready agentic systems

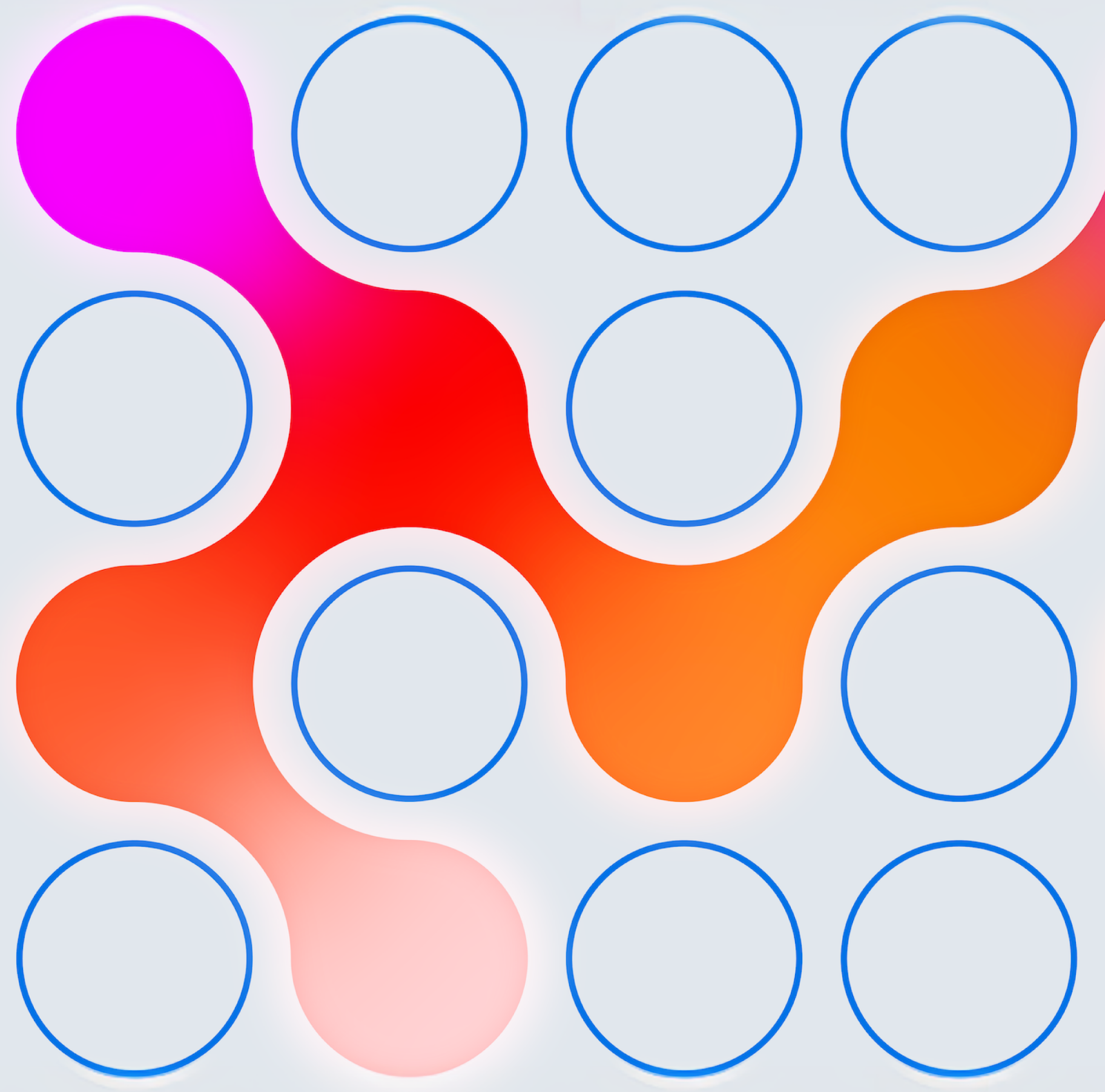


The Agentic AI capability stack

13 capability domains to build production-ready agentic systems



Demo



Agentic AI tools and capabilities in AWS Marketplace

AGENT REASONING

Claude

cohere

Hugging Face

LangChain

Dify

Progress Agentic RAG

composable

Amazon Bedrock

STRANDS AGENTS

EVALUATION & EXPERIMENTATION

LangChain

Langfuse

vellum

Weights & Biases

comet LLM

Patronus AI

deepchecks.

Amazon Bedrock AgentCore (Evaluations)

Amazon SageMaker AI

Amazon CloudWatch

AGENT ROUTING

LangChain

AWS Step Functions

Amazon EventBridge

Amazon Bedrock

TOOLS & INTEGRATION

algolia

mapbox

Deepgram

Kong

Konnect

Tyk

TAILWINDS

portkey

SMARTBEAR.

krakenD

AWS Lambda

Amazon API Gateway

Amazon Bedrock AgentCore (Gateway)

ORCHESTRATION

Temporal

orkes

PREFECT

ASTRONJMER

AWS Step Functions

Amazon Bedrock Agents

Amazon MWA

MEMORY

zilliz

redis

elastic

TiDB Cloud

MongoDB Atlas

Weaviate

yugabyteDB

Pinecone

Amazon OpenSearch

Amazon MemoryDB

Amazon Bedrock AgentCore (Memory)

SECURE IDENTITY

Ping

okta

CYBERARK

Amazon Cognito

AWS Secrets Manager

Amazon Bedrock AgentCore (Identity)

DATA & RETRIEVAL

CONFLUENT

databricks

Fivetran

Collibra

Alation

Bigeye

MONTE CARLO

boomi

Amazon Bedrock Knowledge Bases

Amazon S3

AWS Glue

DEPLOYMENT LIFECYCLE

Pulumi

GitLab

Vercel

LaunchDarkly

Vault

KNIME

AWS CodePipeline

Amazon SageMaker AI

AWS CDK

OBSERVABILITY

DATADOG

honeycomb.io

arize

LangWatch

True Watch

INSTANA

logz.io

Amazon CloudWatch

AWS X-Ray

AWS Distro for OpenTelemetry

SECURITY HARDENING

snyk

Protect AI

Prompt

tailscale

Amazon GuardDuty

AWS Security Hub

Amazon Inspector

SCALING & COST GOVERNANCE

ClickHouse

NVIDIA AI ENTERPRISE

Modal

cast.ai

baseten

anyscale

hydrolix

StormForge

Amazon EKS

AWS Fargate

Amazon Bedrock AgentCore (Runtime)

GUARDRAILS & SAFETY

Claude

Guardrails AI

Arthur

Amazon Bedrock Guardrails

Amazon Bedrock AgentCore (Policy)

2,800+

Agentic AI tools and capabilities

To help you build, deploy, and operate agentic systems on AWS—available through your AWS account.

aws marketplace

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

31

Why AWS Marketplace?

Streamline product discovery, procurement, and deployment.



AWS Marketplace offers pricing flexibility

Flexible consumption models

Ease into commitments

Free trials

Get started with a free trial before committing to purchase.



Bring your own license

Migrate to AWS with your existing product licenses. Intended for pre-existing customers.



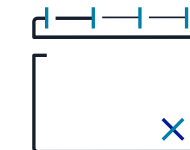
Usage-based subscriptions

Pay for software and compute capacity by the second or hour with no long-term commitments.



Contracts

Make a monthly, annual, or multi-year payment and receive a discount.



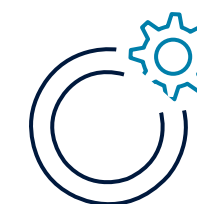
Private offers

Negotiate a custom price with a seller or Channel Partner. Accept the offer in AWS Marketplace.



Flexible payment scheduler

Set a custom billing frequency with your seller.



Key takeaways

Domain specialization is engineering, not configuration. Orchestration at scale is the next layer.

The four levers are active concerns

System prompt · corpus · tools · guardrails.
Not defaults. Deliberately tuned per domain.

Threshold first, then build

Customer-facing demands 90–95%. Safety-critical approaches 99%. The target dictates the effort.

Infrastructure > prompt for what matters

Safety, liability, and PII live in Amazon Bedrock Guardrails—not prose the model can reason around.

Geospatial and voice are tool-use problems

Let Location Service, Mapbox, Transcribe, Polly, and Deepgram do the computation the model cannot.

Golden datasets must be domain-representative

Hand-picked demos lie. Real distribution reveals where the agent will actually fail.



Try it for yourself

Visit the GitHub repository to get the code from today's demo. Start testing immediately following the workshop.

Next steps



Domain specific agent applications

Read the companion guide: Transform general-purpose agents into domain specialists using four architectural levers – applied across three production domains on AWS.



Building agentic systems on AWS

Explore the 14-module workshop series. Learn to design, orchestrate, and scale multi-agent systems—from foundational patterns to production deployment—with structured tutorials and vetted AWS Marketplace integrations.

Thank you!